

ROS 2 Jazzy Coursework Worksheet

URDF → Gazebo → RViz → ros2_control → Teleop → SLAM → Nav2 → Simple Commander → Object Detection → Pick & Place

1. URDF & Xacro

Objectives

- Build a complete robot model: base, wheels, arm, gripper, sensors.
- Use URDF for structure and Xacro for modularity.
- Include visual, collision, and inertial elements for each link.
- Verify the TF tree using robot_state_publisher and RViz.

Key Tasks

- Create links and joints for mobility and manipulation.
- Add sensors (camera, lidar, depth camera).
- Convert URDF → Xacro using macros.
- Visualise the model in RViz.

References

- URDF basics:
<https://docs.ros.org/en/jazzy/Tutorials/Intermediate/URDF/Building-a-Visual-Robot-Model-with-URDF-from-Scratch.html>
 - Movable robot:
<https://docs.ros.org/en/jazzy/Tutorials/Intermediate/URDF/Building-a-Movable-Robot-Model-with-URDF.html>
 - Using Xacro:
<https://docs.ros.org/en/jazzy/Tutorials/Intermediate/URDF/Using-Xacro-to-Clean-Up-a-URDF-File.html>
 - robot_state_publisher:
<https://docs.ros.org/en/jazzy/Tutorials/Intermediate/URDF/Using-URDF-with-Robot-State-Publisher-py.html>
-

2. Gazebo Simulation & ros2_control

Objectives

- Load your Xacro-based robot into Gazebo.
- Enable wheel and joint control via ros2_control.
- Validate robot motion inside the simulator.

Key Tasks

- Add <ros2_control> elements to Xacro for actuators.
- Launch simulation with controllers.
- Test using /cmd_vel and joint command topics.

Reference

https://control.ros.org/jazzy/doc/ros2_control_demos/example_2/doc/userdoc.html

3. Teleoperation

Objectives

- Drive the robot via keyboard or joystick.
- Test basic mobility before enabling SLAM or Nav2.

Key Tasks

- Launch teleop node.
 - Confirm /cmd_vel is routed to your diff_drive controller.
 - Ensure motion looks correct in Gazebo.
-

4. SLAM (slam_toolbox)

Objectives

- Build a consistent map of the harmonic Gazebo world.
- Use SLAM with lidar or depth/camera-based techniques.
- Save map for navigation.

Key Tasks

- Launch slam_toolbox in online mode.
- Teleoperate to explore the environment.
- Save map for later use with Nav2.

Reference

https://docs.ros.org/en/ros2_packages/jazzy/api/slam_toolbox/

5. Navigation (Nav2)

Objectives

- Use saved map for autonomous navigation.
- Configure costmaps, planner, controller, and BT navigator.
- Validate goal reaching via RViz.

Key Tasks

- Bring up Nav2 with your map.
 - Send navigation goals from RViz.
 - Verify obstacle inflation and path planning.
-

6. Waypoint Navigation (Nav2 Simple Commander)

Objectives

- Control navigation behaviour from a Python script.
- Navigate robot through a sequence of waypoints.

Key Tasks

- Initialise BasicNavigator.
- Send multiple goals sequentially.
- Confirm robot reaches each waypoint.

Reference

https://docs.ros.org/en/ros2_packages/jazzy/api/nav2_simple_commander/

7. Ball Detection (No ModelState Access Allowed)

Objectives

- Detect spheres in the Gazebo harmonic world *without* reading `/model_states`.
- Estimate sphere poses in robot or map frame.
- Provide usable target coordinates for pick & place.

High-Level Approaches (no implementation details)

- **RGB camera:** colour-based segmentation or 2D object detection.
 - **Depth camera:** detect round objects using depth discontinuities or clustering.
 - **Point cloud processing:** remove ground plane, cluster objects, find sphere-sized clusters.
 - **Machine learning:** small object detection model; convert 2D detections into 3D via depth.
-

8. Pick & Place Pipeline (Concept-Level Only)

Objectives

- Navigate to the ball.
- Use the arm to execute a simple grasp.
- Transport object to target zone and release.

General Steps

1. Navigate robot near the detected ball.
 2. Align robot orientation for grasp.
 3. Position arm above ball using IK.
 4. Close gripper to pick.
 5. Navigate to target area.
 6. Place ball and release.
-

9. Verification Checklist

Robot Description

- ☐ URDF displays correctly in RViz
- ☐ Xacro macros reduce repeated elements
- ☐ All links have inertial/collision data

Simulation

- ☐ Gazebo loads robot with functioning controllers
- ☐ Wheels and joints respond to commands
- ☐ TF tree has no broken links

Teleoperation

- ☐ Teleop keyboard/joystick drives robot
- ☐ Motion looks realistic

SLAM

- ☐ Online mapping works
- ☐ Map saved with correct structure
- ☐ Frames aligned (map, odom, base_link)

Navigation

- ☐ Nav2 starts without errors
- ☐ Goals in RViz reachable
- ☐ Costmaps behave correctly

Simple Commander

- ☐ Waypoints sent from script
- ☐ Robot reaches each waypoint

Ball Detection

- ☐ Perception detects ball pose without /model_states
- ☐ Pose estimate stable enough for navigation

Pick & Place

- ☐ Robot approaches ball successfully
 - ☐ Grasp is stable
 - ☐ Ball placed correctly in target zone
-

10. Suggested Project Folder Structure

ros2_ws/

└─ src/

└─ my_bot_description/ # URDF + Xacro + meshes

└─ my_bot_gazebo/ # Gazebo world + spawn files

└─ my_bot_control/ # ros2_control configs + controllers

└─ my_bot_teleop/ # Keyboard teleoperation

└─ my_bot_slam/ # slam_toolbox + mapping launch

└─ my_bot_navigation/ # Nav2 + maps + Simple Commander

└─ my_bot_perception/ # Ball detection (camera/depth)